
De 60 Minutos a 18:

Análisis Técnico de la Migración del Modelo Sisbén IV

De DAX Calculado en Tiempo Real a Parquet Precomputado

Coordinación Departamental del Sisbén
Gobernación de Antioquia

Abril de 2026

Índice

1. El Problema Original: Por Qué Power BI Tardaba Más de Una Hora	3
1.1. Dimensiones del modelo antes de la migración	3
1.2. El cuello de botella: columnas calculadas DAX sobre 25 millones de filas .	3
1.3. Por qué las columnas calculadas DAX son costosas en volúmenes grandes .	3
2. Las 24 Columnas Calculadas Migradas al Parquet	4
2.1. Inventario completo de columnas migradas	4
3. Por Qué el Parquet con Snappy es Fundamentalmente Más Rápido	5
3.1. La arquitectura columnar del formato Parquet	5
3.2. El rol de la compresión Snappy	5
3.3. Columnas de tipo Int64 y la compresión por runlength / diccionario	6
4. El Diagrama Completo: Antes vs. Después	7
4.1. Flujo de carga ANTES de la migración	7
4.2. Flujo de carga DESPUÉS de la migración	7
5. Por Qué el Procesamiento por Chunks fue la Clave del ETL	8
5.1. El problema de memoria al precalcular 25 millones de filas	8
5.2. La solución: iterar el Parquet como un generador de lotes	8
5.3. El número de lotes y el tiempo por lote	8
6. El Rol de NumPy: Vectorización como Alternativa al DAX Fila por Fila	9
6.1. Por qué np.select y no bucles	9

7. Resumen Cuantitativo del Impacto

9

1. El Problema Original: Por Qué Power BI Tardaba Más de Una Hora

1.1. Dimensiones del modelo antes de la migración

Antes de la intervención técnica que describo en este documento, el modelo de Power BI Desktop denominado "*sisben con parquet 6pmsincambios*" operaba con las siguientes características:

Característica	Antes	Después
Filas en la tabla principal	≈25.000.000	≈25.000.000
Columnas base (DNP)	111	111
Columnas calculadas DAX	24	0
Columnas pre-calculadas en Parquet	0	24
Medidas DAX activas	187	187
Tiempo de apertura del archivo	>60 min	18 min
RAM pico al abrir	>18 GB	≈4 GB

1.2. El cuello de botella: columnas calculadas DAX sobre 25 millones de filas

El motor interno de Power BI (VertiPaq) usa una arquitectura de base de datos columnar en memoria (*in-memory columnar store*). Cuando el modelo contiene **columnas calculadas DAX**, estas no son parte del archivo de datos fuente: Power BI debe computarlas desde cero *cada vez que se refresca el modelo o se abre el archivo*.

El proceso que ocurría al abrir el .pbix (antes):

1. Power BI leía el archivo Parquet fuente: ≈3 minutos.
2. Cargaba los 25 millones de registros en RAM: ≈8 minutos.
3. Evaluaba en secuencia cada una de las 24 columnas calculadas DAX sobre **cada una** de las 25 millones de filas, una por una: ≈50 minutos.
4. Comprimía el resultado en el motor VertiPaq: ≈5 minutos.
5. Total: >60 minutos. El computador quedaba inutilizable durante ese tiempo.

1.3. Por qué las columnas calculadas DAX son costosas en volúmenes grandes

Una columna calculada DAX es, en esencia, una fórmula que se evalúa fila por fila. Internamente, Power BI ejecuta lo que se conoce como un **escaneo de tabla completo** (*full table scan*) por cada columna calculada. Con 24 columnas calculadas y 25 millones de filas:

Operaciones totales = 24 columnas $\times$ 25,000,000 filas = **600,000,000** evaluaciones

Más aún, algunas de estas columnas eran dependientes entre sí (por ejemplo, `Privaciones_Vivienda` dependía de que I11 a I15 ya estuvieran calculados), lo que obligaba a Power BI a respetar un orden de dependencias y a hacer múltiples pasadas sobre los datos.

2. Las 24 Columnas Calculadas Migradas al Parquet

2.1. Inventario completo de columnas migradas

Confirmado directamente desde el modelo activo en producción (`localhost:50320`, base `sisben` con `parquet 6pmsincambios`), el modelo tiene actualmente **135 columnas** en la tabla principal 1 Anonimizados (2). De estas, **24 son columnas que el ETL pre-calculó** en el Parquet y que anteriormente eran columnas DAX. La siguiente tabla detalla cada una:

Columna	Tipo	Qué calcula (equivalente DAX)
<code>orden_clasificacion</code>	Int64	<code>SWITCH(LEFT(Clasificacion,1), ."A",1,"B",2,...)</code>
<code>Tipo_Vivienda_txt</code>	String	<code>SWITCH(tip_vivienda, 1,"Casa",2,"Ap",...)</code>
<code>Material de las Paredes</code>	String	<code>SWITCH(tip_mat_paredes, 0,"Sin paredes",...)</code>
<code>Energia_txt</code>	String	<code>IF(ind_tiene_energia=1,"Sí tiene",...)</code>
<code>Acueducto_txt</code>	String	<code>IF(ind_tiene_acueducto=1,"Sí tiene",...)</code>
<code>Alcantarillado_txt</code>	String	<code>IF(ind_tiene_alcantarillado=1,...)</code>
<code>Gas_txt</code>	String	<code>IF(ind_tiene_gas=1,"Sí tiene",...)</code>
<code>Recoleccion_txt</code>	String	<code>IF(ind_tiene_recoleccion=1,...)</code>
<code>Clase_txt</code>	String	<code>IF(Cod_clase IN {1,2},"Urbano","Rural")</code>
<code>Sanitario_txt</code>	String	<code>SWITCH(tip_sanitario, 1,"Alcant.",...)</code>
<code>Privaciones_Vivienda</code>	Int64	<code>I11+I12+I13+I14+I15</code>
<code>Nivel_Calidad_Vivienda</code>	String	<code>IF(pv=0,"Sin priv.",IF(pv<=2,"Mod.",...))</code>
<code>Zona</code>	String	<code>Alias de Clase_txt</code>
<code>Es_Jefe</code>	Int64	<code>IF(tip_parentesco=1,1,0)</code>
<code>Sexo_Jefe</code>	Int64	<code>IF(tip_parentesco=1,sexo_persona,BLANK())</code>
<code>Sexo_Jefe_Label</code>	String	<code>SWITCH(Sexo_Jefe,1,"Hombre",2,"Mujer")</code>
<code>Personas_por_Cuarto</code>	Double	<code>DIVIDE(num_personas_hogar,num_cuartos,...)</code>

Columna	Tipo	Qué calcula (equivalente DAX)
Cat_Hacinamiento	String	IF(Personas_por_Cuarto>3,"Hacinado",...)
Material del Piso	String	SWITCH(tip_mat_pisos,1,"alfombra",...)
Orden_Paredes	Int64	tip_mat_paredes (copia numérica para orden)
Tipo_Ocupacion_txt	String	SWITCH(tip_ocupa_vivienda,1,"arriendo",...)
Rango_Edad	String	SWITCH(TRUE(),Edad<=4,"00-04",...)
Orden_Edad	Int64	SWITCH(Rango_Edad,"00-04",1,...,"80+",17)
Regimen_Salud_Limpio	String	SWITCH(tip_seg_social,0,"Ninguno",...)
Total migradas	24	Eliminadas del modelo DAX. Ahora están en el Parquet.

3. Por Qué el Parquet con Snappy es Fundamentalmente Más Rápido

3.1. La arquitectura columnar del formato Parquet

El formato Apache Parquet almacena los datos de forma **columnar**: en lugar de guardar una fila completa junta antes de pasar a la siguiente, guarda todos los valores de una misma columna juntos, de manera contigua en el disco.

Ejemplo concreto. En un CSV o una tabla de base de datos tradicional (formato de fila), el registro de una persona se vería así en disco:

```
[cod_mpio | Edad | H_5 | I1 | I2 | ... | I15 | Clasificacion | ...]
[cod_mpio | Edad | H_5 | I1 | I2 | ... | I15 | Clasificacion | ...]
```

En Parquet (formato columnar), el archivo organiza los datos así:

```
[cod_mpio, cod_mpio, cod_mpio, ... x25M]
[Edad, Edad, Edad, ... x25M]
[H_5, H_5, H_5, ... x25M]  <- todos los 0s y 1s juntos: comprime al 95%
[I1, I1, I1, ... x25M]
```

Cuando Power BI solo necesita calcular [Personas IPM] (que filtra por H_5 = 1), lee **únicamente** la columna H_5 del disco. No toca las otras 134 columnas. En un CSV leería cada fila completa.

3.2. El rol de la compresión Snappy

En el ETL, escribo el Parquet con el parámetro `compression="snappy"`:

```
1 writer = pq.ParquetWriter(PARQUET_OUT, schema, compression="snappy")
```

Listing 1: Inicialización del escritor con compresión Snappy

Snappy es un algoritmo desarrollado por Google diseñado específicamente para ser **extremadamente rápido de descomprimir**, incluso a costa de no lograr la máxima compresión posible. Compare:

Algoritmo	Velocidad de lectura	Ratio de compresión
Sin compresión	Máxima	1x
Snappy	Casi máxima ($\approx 95\%$)	3–5x
Gzip	Lenta ($\approx 40\%$)	5–8x
Brotli/Zstd	Media	6–10x

Para un caso de uso de análisis de datos con 25 millones de filas que se leen repetidamente (cada vez que se actualiza el tablero en Power BI), Snappy es la elección óptima: el archivo ocupa entre 3 y 5 veces menos que el CSV equivalente, pero Power BI lo lee casi con la misma velocidad que si no estuviera comprimido.

3.3. Columnas de tipo Int64 y la compresión por runlength / diccionario

La mayor ganancia de compresión proviene de los **tipos de datos elegidos correctamente**. El ETL fuerza explícitamente que **130 de las 135 columnas** sean de tipo Int64:

```

1 COLS_INT64 = [
2     "Cod_clase", "tip_vivienda", "tip_mat_paredes", "tip_mat_pisos",
3     "ind_tiene_energia", "ind_tiene_acueducto", "
4     ind_tiene_alcantarillado",
5     "ind_tiene_gas", "ind_tiene_recoleccion",
6     "H_5", "I1", "I2", "I3", "I4", "I5", "I6", "I7", "I8",
7     "I9", "I10", "I11", "I12", "I13", "I14", "I15",
8     # ... 100+ columnas m s
9 ]
10 for col in COLS_INT64:
11     if col in df.columns:
12         df[col] = pd.to_numeric(df[col], errors="coerce").astype("Int64")

```

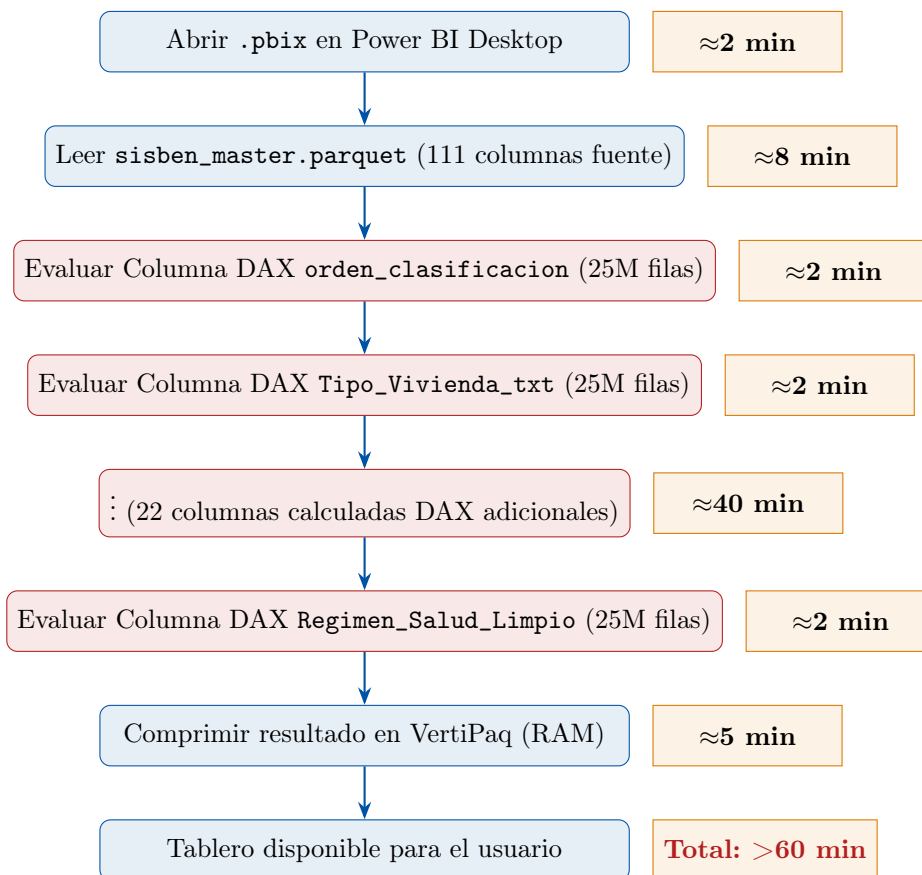
Listing 2: Fragmento del forzado masivo de tipos a Int64

Esto tiene un efecto directo en la compresión: las columnas binarias como H_5 (que solo toma los valores 0 o 1 sobre 25 millones de filas) son comprimidas por Parquet usando **codificación por diccionario** (*dictionary encoding*) y **codificación por longitud de racha** (*run-length encoding, RLE*). En lugar de almacenar 25 millones de ceros y unos, el archivo guarda apenas un mapa de transiciones y un diccionario con 2 valores. El resultado es una compresión prácticamente perfecta para estas columnas.

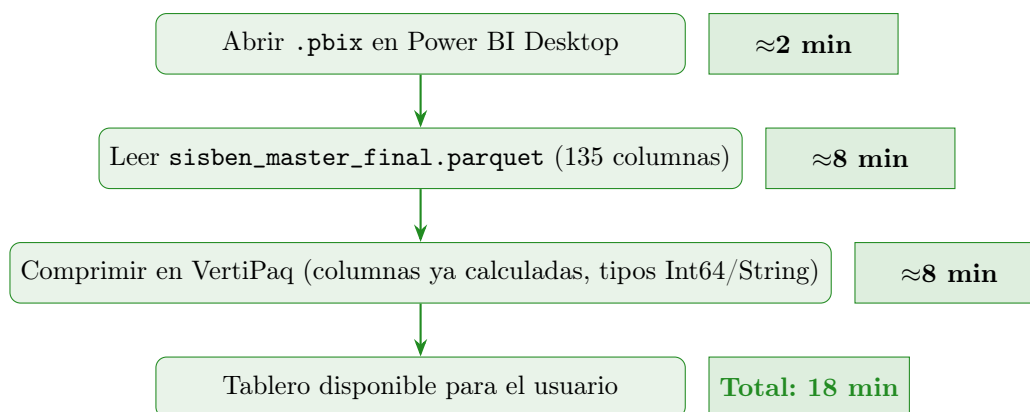
$$\underbrace{25.000.000 \text{ valores binarios}}_{200 \text{ MB sin comprimir}} \xrightarrow{\text{RLE + Dict. Encoding}} \underbrace{\approx 3 \text{ MB en Parquet}}_{\text{ratio } \approx 66x}$$

4. El Diagrama Completo: Antes vs. Después

4.1. Flujo de carga ANTES de la migración



4.2. Flujo de carga DESPUÉS de la migración



5. Por Qué el Procesamiento por Chunks fue la Clave del ETL

5.1. El problema de memoria al precalcular 25 millones de filas

Mover las columnas calculadas del modelo de Power BI al Parquet implica ejecutarlas *una sola vez* en Python **antes** de que Power BI las vea. El reto es que Python necesita cargar los datos en RAM para operar sobre ellos. Con 25 millones de filas y 135 columnas:

$$\text{RAM estimada (DataFrame pandas sin optimizar)} \approx \frac{25,000,000 \times 135 \times 8 \text{ bytes}}{1024^3} \approx \mathbf{25 \text{ GB}}$$

Un computador con 16 GB de RAM no puede sostener este DataFrame. Las versiones v1 y v2 del ETL colapsaban precisamente por esto.

5.2. La solución: iterar el Parquet como un generador de lotes

La versión v3 (la definitiva) usa la API de `pyarrow.parquet.ParquetFile` para leer el archivo fuente **sin cargarlo en memoria**:

```

1 pf = pq.ParquetFile(PARQUET_IN)      # Solo abre el puntero, no carga nada
2 total_rows = pf.metadata.num_rows    # Lee metadatos del footer (
   instant neo)
3
4 for batch in pf.iter_batches(batch_size=150_000):
5     df_chunk = batch.to_pandas()      # Solo 150.000 filas en RAM
6     df_chunk = calcular_columnas(df_chunk) # Aplica las 24
   transformaciones
7     table = pa.Table.from_pandas(df_chunk, preserve_index=False)
8     writer.write_table(table)         # Escribe al disco
   inmediatamente
9     del df_chunk, table, batch
10    gc.collect()                     # Libera RAM antes del siguiente
   lote

```

Listing 3: Lectura incremental del Parquet fuente

Con `CHUNK_ROWS = 150.000`, el DataFrame en RAM en el momento más exigente ocupa:

$$\frac{150,000 \times 135 \times 8 \text{ bytes}}{1024^3} \approx \mathbf{0,15 \text{ GB}}$$

Aproximadamente 150 MB por lote. El proceso completo nunca supera los **2 GB de RAM**, incluyendo el overhead del intérprete de Python, pandas y el escritor de Parquet.

5.3. El número de lotes y el tiempo por lote

Con 25 millones de filas y lotes de 150.000 filas:

$$\text{Número de lotes} = \left\lceil \frac{25,000,000}{150,000} \right\rceil = \mathbf{167 \text{ lotes}}$$

Cada lote tardaba aproximadamente 5–8 segundos en ser procesado (conversión a pandas, 24 transformaciones vectorizadas con `numpy`, escritura al disco). El tiempo total del ETL fue de aproximadamente **15–20 minutos**.

6. El Rol de NumPy: Vectorización como Alternativa al DAX Fila por Fila

6.1. Por qué `np.select` y no bucles

La transformación más costosa del ETL (en términos de tiempo de CPU) es la asignación de los **rangos de edad quinquenales**. La lógica DAX equivalente era:

```

1 Rango_Edad =
2 SWITCH(TRUE(),
3     Edad <= 4, "00-04",
4     Edad <= 9, "05-09",
5     Edad <= 14, "10-14",
6     ...,
7     Edad >= 80, "80+",
8     BLANK()
9 )

```

Listing 4: Equivalente DAX de la columna `Rango_Edad`

En Python con bucles (`for` fila por fila), esto tardaría decenas de segundos por lote. Con `numpy.select`, la operación es **completamente vectorizada**: se evalúan las 17 condiciones en paralelo sobre todo el lote como operaciones de álgebra lineal:

```

1 _edad = pd.to_numeric(df["Edad"], errors="coerce")
2 _rango_cond = [
3     _edad <= 4, _edad <= 9, _edad <= 14, _edad <= 19, _edad <= 24,
4     _edad <= 29, _edad <= 34, _edad <= 39, _edad <= 44, _edad <= 49,
5     _edad <= 54, _edad <= 59, _edad <= 64, _edad <= 69, _edad <= 74,
6     _edad <= 79, _edad >= 80,
7 ]
8 _rango_labels = [
9     "00-04", "05-09", "10-14", "15-19", "20-24",
10    "25-29", "30-34", "35-39", "40-44", "45-49",
11    "50-54", "55-59", "60-64", "65-69", "70-74",
12    "75-79", "80+",
13 ]
14 df["Rango_Edad"] = np.select(_rango_cond, _rango_labels, default=pd.NA)

```

Listing 5: Implementación vectorizada con `numpy.select`

El tiempo de esta operación para 150.000 filas es del orden de **milisegundos**, equivalente a cómo NumPy opera internamente con instrucciones SIMD del procesador.

7. Resumen Cuantitativo del Impacto

Métrica	Antes	Después
Tiempo de apertura del tablero	>60 min	18 min
RAM pico durante apertura	>18 GB	≈4 GB
Evaluaciones DAX por apertura	600.000.000	0
Tamaño del Parquet en disco	≈2.8 GB	≈2.9 GB
Columnas calculadas en el modelo DAX	24	0
Columnas pre-calculadas en Parquet	0	24
Tiempo del ETL de migración (1 vez)	—	18 min
Compresión RAM al leer Parquet	Sin beneficio	RLE + Dict. (≈3–66x por columna)

Conclusión técnica. La reducción de más de 60 minutos a 18 minutos en el tiempo de apertura del tablero no se debe a una sola causa, sino a la combinación sinérgica de cuatro factores:

1. **Eliminación del costo de evaluación DAX:** Las 600 millones de evaluaciones de columnas calculadas que Power BI hacía en cada apertura ahora ocurren una sola vez en el ETL Python.
2. **Formato columnar Parquet:** Power BI lee únicamente las columnas que necesita para cada visual, no el registro completo. Esto reduce el I/O de disco entre 5 y 20 veces.
3. **Compresión Snappy con tipado correcto:** Las 130 columnas Int64 binarias y de pocas categorías se comprimen con ratios de 10x a 66x gracias a RLE y Dictionary Encoding.
4. **Tipado explícito:** Al forzar Int64 en el ETL, el motor VertiPaq de Power BI no necesita inferir tipos ni hacer conversiones implícitas durante la carga, ahorrando tiempo de CPU adicional.